CSI3007 - Advanced Python Programming

Lab Practice

NAME : RAJAVELMURUGAN S

REG_NO : 23 MID0018

FACULTY : SHARMILA BANU K

LAB SLOT : L31+L32+L49+L50

Problem #1: Design a Python class called Student to track a student's academic performance.

Requirements:

Constructor (_init_)

Accepts two parameters: name (string) and student_id (string or int).

Initializes an empty list scores to store the student's marks.

Instance Variables:

name → student's name

student_id → student's unique ID

scores → list of numeric scores (initially empty)

Methods:

add_score(score) → Appends a new score (int/float) to the scores list.

calculate_average() → Returns the average of all scores. If no scores exist, return 0.

get_final_grade() → Uses the average to determine result:

"Pass" if average ≥ 60

"Fail" otherwise

Test Cases

1. Object Creation

```
student1 = Student("Alice", 101)
print(student1.name)       # Expected: Alice
print(student1.student_id)  # Expected: 101
print(student1.scores)      # Expected: []
```

2. Adding Scores

```
student1.add_score(75)
student1.add_score(85)
student1.add_score(55)
print(student1.scores)      # Expected: [75, 85, 55]
```

3. Calculate Average

print(student1.calculate_average())

Expected: (75 + 85 + 55) / 3 = 71.67

4. Final Grade (Pass Case)

print(student1.get_final_grade())

Expected: Pass (average = 71.67 ≥ 60)

5. Final Grade (Fail Case)

student2 = Student("Bob", 102)

student2.add_score(40)

student2.add_score(50)

print(student2.calculate_average()) # Expected: 45.0

print(student2.get_final_grade())   # Expected: Fail

6. Empty Scores Case

student3 = Student("Charlie", 103)

print(student3.calculate_average()) # Expected: 0

print(student3.get_final_grade())   # Expected: Fail

CODE:

```
class Student:

    # Initialize student details
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.scores = []

    # Store a new mark
    def add_score(self, score):
        self.scores.append(score)

    # Find the average of all marks
    def calculate_average(self):
        if len(self.scores) == 0:
            return 0
        return sum(self.scores) / len(self.scores)
```

```
    # Check whether the student passed or failed
    def get_final_grade(self):
        if self.calculate_average() >= 60:
            return "Pass"
        return "Fail"


# Create first student
student1 = Student("Alice", 101)

# Display student details
print(student1.name)
print(student1.student_id)
print(student1.scores)

# Add marks
student1.add_score(75)
student1.add_score(85)
student1.add_score(55)

# Display marks
print(student1.scores)

# Display average mark
print(round(student1.calculate_average(), 2))

# Display result
print(student1.get_final_grade())

# Create second student
student2 = Student("Bob", 102)

# Add marks
student2.add_score(40)
student2.add_score(50)

# Display average and result
print(student2.calculate_average())
print(student2.get_final_grade())

# Create third student
student3 = Student("Charlie", 103)

# Check average and result when no marks are added
print(student3.calculate_average())
print(student3.get_final_grade())
```

OUTPUT:

```
Alice
101
[]
[75, 85, 55]
71.67
Pass
45.0
Fail
0
Fail
```

Problem #2: Design a Python class called Gene to represent a DNA sequence.

The class should:

Store a gene's ID and sequence.

Provide methods to:

Print the gene's ID and sequence.

Count the occurrence of a specific base (A, T, G, C).

Calculate the GC content (fraction of G and C bases).

Get and set the sequence, ensuring that RNA bases (U) are not allowed.

Test Cases

1. Object Creation

```
gene1 = Gene(101, "ATGCGT")
# Expected output (from constructor prints):
# I'm a new Gene object!
# My constructor got a param: 101
# Assign that param to my id instance variable...
# Similarly, assigning to my sequence instance variable...
```

2. Print ID and Sequence

```
gene1.print_id()
# Expected: My id is: 101
gene1.print_sequence()
# Expected: My sequence is: ATGCGT
```

3. Base Composition

```
print(gene1.base_composition("A"))
# Expected: 1
print(gene1.base_composition("G"))
# Expected: 2
print(gene1.base_composition("T"))
# Expected: 1
```

4. GC Content

print(gene1.gc_content())

Sequence: ATGCGT → G=2, C=1 → GC count=3

Length=6 → Expected: 3/6 = 0.5

5. Get Sequence

print(gene1.get_seq())

Expected: ATGCGT

6. Set Sequence (Valid DNA)

gene1.set_seq("ATATGC")

print(gene1.get_seq())

Expected: ATATGC

7. Set Sequence (Invalid RNA)

gene1.set_seq("AUGC")

Expected: AssertionError → "Sorry, no RNA allowed."

CODE:

```
class Gene:

    # Initialize gene details
    def __init__(self, gene_id, sequence):
        print("I'm a new Gene object!")
        print("My constructor got a param:", gene_id)
        print("Assign that param to my id instance variable...")
        self.gene_id = gene_id
        print("Similarly, assigning to my sequence instance variable...")
        self.sequence = sequence

    # Display gene ID
    def print_id(self):
        print("My id is:", self.gene_id)

    # Display DNA sequence
    def print_sequence(self):
        print("My sequence is:", self.sequence)

    # Count a specific DNA base
    def base_composition(self, base):
        return self.sequence.count(base)
```

```
    # Find GC content
    def gc_content(self):
        gc = self.sequence.count("G") + self.sequence.count("C")
        return gc / len(self.sequence)

    # Return the sequence
    def get_seq(self):
        return self.sequence

    # Update the sequence
    def set_seq(self, new_seq):
        assert "U" not in new_seq, "Sorry, no RNA allowed."
        self.sequence = new_seq


# Create a Gene object
gene1 = Gene(101, "ATGCGT")

# Print gene ID
gene1.print_id()

# Print DNA sequence
gene1.print_sequence()

# Count A
print(gene1.base_composition("A"))

# Count G
print(gene1.base_composition("G"))

# Count T
print(gene1.base_composition("T"))

# Print GC content
print(gene1.gc_content())

# Get the sequence
print(gene1.get_seq())

# Set a valid DNA sequence
gene1.set_seq("ATATGC")

# Print updated sequence
print(gene1.get_seq())

# Try an invalid RNA sequence
gene1.set_seq("AUGC")
```

OUTPUT:

```
I'm a new Gene object!
My constructor got a param: 101
Assign that param to my id instance variable...
Similarly, assigning to my sequence instance variable...
My id is: 101
My sequence is: ATGCGT
1
2
2
0.5
ATGCGT
ATATGC
```

Fix Code

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[5], line 70
     67 print(gene1.get_seq())
     69 # Try an invalid RNA sequence
---> 70 gene1.set_seq("AUGC")

Cell In[5], line 35, in Gene.set_seq(self, new_seq)
     34 def set_seq(self, new_seq):
---> 35     assert "U" not in new_seq, "Sorry, no RNA allowed."
     36     self.sequence = new_seq

AssertionError: Sorry, no RNA allowed.
```

Error Handled using exception:

```python
# Set a valid DNA sequence
gene1.set_seq("ATATGC")

# Print the updated sequence
print(gene1.get_seq())

# Try an invalid RNA sequence
try:
    gene1.set_seq("AUGC")

# Print the error message
except AssertionError as e:
    print(e)
```

```
I'm a new Gene object!
My constructor got a param: 101
Assign that param to my id instance variable...
Similarly, assigning to my sequence instance variable...
My id is: 101
My sequence is: ATGCGT
1
2
2
0.5
ATGCGT
ATATGC
Sorry, no RNA allowed.
```

23MID0018 RAJAVELMURUGN S

Threading using single thread

```
import threading #import threading
import time # import time module
start = time.perf_counter()
def do_something():
    print('sleeping')
    time.sleep(2)
    print('Done sleeping...')
t1 = threading.Thread(target=do_something)
t1.start() # start the thread
t1.join()# join the thread
```

```
sleeping
Done sleeping...
```

This code creates and executes a background thread (t1) to run a function that sleeps for 2 seconds, while forcing the main program to pause and wait for it to finish before continuing

Thread using for loop

```
import threading #import threading
import time # import time module
start = time.perf_counter()
def do_something():
    print('sleeping')
    time.sleep(2)
    print('Done sleeping...')
threads=[]
for _ in range(10):
    t1 = threading.Thread(target=do_something)
    t1.start() # start the thread
    threads.append(t1)
for i in threads:
    i.join() # join the thread
finish = time.perf_counter()
print(f'Finished in {round(finish - start,2)} seconds')
```

```
sleeping
sleeping
sleeping
```

```
sleeping
sleeping
sleeping
sleeping
sleeping
sleeping
sleeping
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 2.0 seconds
```

concurrently runs 10 background threads that all sleep at the same time, completing the entire process in just 2 seconds instead of taking 20 seconds.

Thread using for loop and passing arguments

```
import threading #import threading
import time # import time module
start = time.perf_counter() # count start time
def do_something(sec):
    print('sleeping')
    time.sleep(sec)
    print('Done sleeping...')
threads=[] # list to append threads
for _ in range(10):
    t1 = threading.Thread(target=do_something , args =[2]) # create
threads using loop
    t1.start() # start the thread
    threads.append(t1) # append thread to list
for i in threads:
    i.join() # join the thread
finish = time.perf_counter()
print(f'Finished in {round(finish - start,2)} seconds')


sleeping
sleeping
sleeping
sleeping
sleeping
```

```
sleeping
sleeping
sleeping
sleeping
sleeping
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 2.0 seconds
```

This code correctly passes arguments to multiple threads, resulting in all 10 threads sleeping simultaneously and finishing the entire program in exactly 2.0 seconds

ThreadPoolExecutor Single Task Execution

```
import concurrent.futures
import time
start = time.perf_counter()
def do_something(sec):
    print(f'sleeping{sec} second (s)')
    time.sleep(sec)
    return 'Done sleeping...'
with concurrent.futures.ThreadPoolExecutor() as executor: # use
threadPoolExecutor for collection of threads
    f1 = executor.submit(do_something,1.5)
    print(f1.result())
finish = time.perf_counter()
print(f'Finished in {round(finish - start,2)} seconds')

sleeping1.5 second (s)
Done sleeping...
Finished in 1.5 seconds
```

code utilizes Python's modern ThreadPoolExecutor to submit a single task to a background thread pool instead of manually managing threads. The f1.result() method pauses the main program, waiting for the 1.5-second sleep to complete before retrieving the return value and printing the total execution time.

Concurrent Task Execution with as_completed()

```
import concurrent.futures # import concurrent futures for
ThreadpoolExecuter
import time
start = time.perf_counter()
def do_something(sec):
    print(f'sleeping{sec} second (s)')
    time.sleep(sec) # make sleep
    print('Done sleeping...')
with concurrent.futures.ThreadPoolExecutor() as executor:  # use
threadPoolExecutor for collection of threads
    res = [executor.submit(do_something,1.5) for _ in range(2)] # use
list comprehension for creating multiple threads sends to future
object
    for f in concurrent.futures.as_completed(res): # gives future when
task finishes
        print(f.result())
finish = time.perf_counter()
print(f'Finished in {round(finish - start,2)} seconds')
```

```
sleeping1.5 second (s)
sleeping1.5 second (s)
Done sleeping...
Done sleeping...
None
None
Finished in 1.5 seconds
```

two tasks to a thread pool simultaneously using a list comprehension and wraps them as Future objects. The as_completed() iterator then yields each thread's result the exact moment it finishes, ensuring both tasks process concurrently in roughly 1.5 seconds total.

Asynchronous Future Resolution with Return Values

```
import concurrent.futures # import concurrent futures for
ThreadpoolExecuter
import time
start = time.perf_counter()
def do_something(sec):
    print(f'sleeping{sec} second (s)')
    time.sleep(sec)
    return'Done sleeping...'
with concurrent.futures.ThreadPoolExecutor() as executor:
    res = [executor.submit(do_something,1.5) for _ in range(2)] # use
list comprehension for creating multiple threads sends to future
```

```
object
    for f in concurrent.futures.as_completed(res): # gives future when
task finishes
        print(f.result())
finish = time.perf_counter()
print(f'Finished in {round(finish - start,2)} seconds')
```

```
sleeping1.5 second (s)
sleeping1.5 second (s)
Done sleeping...
Done sleeping...
Finished in 1.5 seconds
```

two tasks concurrently in a thread pool and uses f.result() to print the string returned by the function ('Done sleeping...'). Because the threads run at the same time and return values upon completion, the entire script finishes execution in approximately 1.5 seconds.

Mapping Multiple Inputs to Parallel Threads

```
import concurrent.futures # import concurrent futures for
ThreadpoolExecuter
import time
start = time.perf_counter()
def do_something(sec):
    print(f'sleeping{sec} second (s)')
    time.sleep(sec)
    return'Done sleeping...'
with concurrent.futures.ThreadPoolExecutor() as executor:  # use
threadPoolExecutor for collection of threads
    sec= [4,5,6,7]
    res = executor.map(do_something,sec) # used to map diff inputs to
function
    for result in res:
        print(result)
finish = time.perf_counter()
print(f'Finished in {round(finish - start,2)} seconds')
```

```
sleeping4 second (s)
sleeping5 second (s)
sleeping6 second (s)
sleeping7 second (s)
Done sleeping...
Done sleeping...
Done sleeping...
Done sleeping...
Finished in 7.0 seconds
```

executor.map() to automatically spawn four parallel threads, processing the array of sleep times [4, 5, 6, 7] concurrently. Because all tasks run simultaneously in the background, the total execution time is determined by the longest task, finishing in approximately 7.0 seconds instead of 22 seconds.

Web Scraping and Media Downloading (MANUALLY)

```
import time
import requests
vid_url = ['https://pixabay.com/videos/spiral-particles-wave-intro-
217490/',
           'https://pixabay.com/videos/flash-wave-surge-particles-
199581/',
           'https://pixabay.com/videos/intro-wave-flash-light-
217642/',
           'https://pixabay.com/videos/intro-circles-flash-wave-
231786/',
           'https://pixabay.com/videos/explosion-fireworks-flash-
start-175299/',
           'https://pixabay.com/videos/woman-sample-laboratory-
quality-197485/',
           'https://pixabay.com/videos/sparks-particles-sample-
background-212966/',
           'https://pixabay.com/videos/praya-de-paradiso-drone-
cinematic-35540/',
           'https://pixabay.com/videos/microscope-medical-laboratory-
5445/',
           'https://pixabay.com/videos/exploration-galaxy-generation-
ship-130414/',
           'https://pixabay.com/videos/clock-time-minutes-green-
screen-149068/',
           'https://pixabay.com/videos/laboratory-glassware-
microbiology-262189/',
           'https://pixabay.com/videos/sphere-space-spacecraft-
130108/',
           'https://pixabay.com/videos/information-window-information-
plate-151346/',
           'https://pixabay.com/videos/red-kaleidoscope-sample-
background-108069/',
           'https://pixabay.com/videos/purple-green-lines-abstraction-
124825/',
           'https://pixabay.com/videos/circles-kaleidoscope-sample-
124140/',
           'https://pixabay.com/videos/particles-abstract-beautiful-
bokeh-34965/'
```

```
]
t1=time.perf_counter()
for i in vid_url:
    vid_bytes= requests.get(i).content # Download the raw data from
the link
    vid_name= i.split('/')[4] # Extract the video name from the URL
text
    vid_name = f'{vid_name}.mp4'
    with open (vid_name,'wb') as vid_file: # Open a local file and
save the downloaded data into it
        vid_file.write(vid_bytes)
        print(f'{vid_name} was downloaded')
t2 = time.perf_counter()
print(f'Finished in {t2-t1} seconds') # Show the total time taken
```

```
spiral-particles-wave-intro-217490.mp4 was downloaded
flash-wave-surge-particles-199581.mp4 was downloaded
intro-wave-flash-light-217642.mp4 was downloaded
intro-circles-flash-wave-231786.mp4 was downloaded
explosion-fireworks-flash-start-175299.mp4 was downloaded
woman-sample-laboratory-quality-197485.mp4 was downloaded
sparks-particles-sample-background-212966.mp4 was downloaded
praya-de-paradiso-drone-cinematic-35540.mp4 was downloaded
microscope-medical-laboratory-5445.mp4 was downloaded
exploration-galaxy-generation-ship-130414.mp4 was downloaded
clock-time-minutes-green-screen-149068.mp4 was downloaded
laboratory-glassware-microbiology-262189.mp4 was downloaded
sphere-space-spacecraft-130108.mp4 was downloaded
information-window-information-plate-151346.mp4 was downloaded
red-kaleidoscope-sample-background-108069.mp4 was downloaded
purple-green-lines-abstraction-124825.mp4 was downloaded
circles-kaleidoscope-sample-124140.mp4 was downloaded
particles-abstract-beautiful-bokeh-34965.mp4 was downloaded
Finished in 1.5705148720007855 seconds
```

downloads 18 video pages sequentially using a for loop, extracting raw content with requests.get() and saving each file locally as an MP4. Because it processes each URL one after the other, the program spends most of its time waiting on network responses (I/O-bound), resulting in a slow total execution time.

Multi-Threaded Concurrent Web Downloading

```
import time
import concurrent.futures
import requests

vid_url = ['https://pixabay.com/videos/spiral-particles-wave-intro-
217490/',
```

```
            'https://pixabay.com/videos/flash-wave-surge-particles-
199581/',
            'https://pixabay.com/videos/intro-wave-flash-light-
217642/',
            'https://pixabay.com/videos/intro-circles-flash-wave-
231786/',
            'https://pixabay.com/videos/explosion-fireworks-flash-
start-175299/',
            'https://pixabay.com/videos/woman-sample-laboratory-
quality-197485/',
            'https://pixabay.com/videos/sparks-particles-sample-
background-212966/',
            'https://pixabay.com/videos/praya-de-paradiso-drone-
cinematic-35540/',
            'https://pixabay.com/videos/microscope-medical-laboratory-
5445/',
            'https://pixabay.com/videos/exploration-galaxy-generation-
ship-130414/',
            'https://pixabay.com/videos/clock-time-minutes-green-
screen-149068/',
            'https://pixabay.com/videos/laboratory-glassware-
microbiology-262189/',
            'https://pixabay.com/videos/sphere-space-spacecraft-
130108/',
            'https://pixabay.com/videos/information-window-information-
plate-151346/',
            'https://pixabay.com/videos/red-kaleidoscope-sample-
background-108069/',
            'https://pixabay.com/videos/purple-green-lines-abstraction-
124825/',
            'https://pixabay.com/videos/circles-kaleidoscope-sample-
124140/',
            'https://pixabay.com/videos/particles-abstract-beautiful-
bokeh-34965/'

]

def download_video(url): # Worker function to handle an individual URL
download
    start = time.perf_counter()
    vid_bytes = requests.get(url).content
    vid_name = url.split('/')[4]     # Extract name from URL text
string
    vid_name = f'{vid_name}.mp4'
    with open(vid_name, 'wb') as vid_file:   # Write binary data to
local file
        vid_file.write(vid_bytes)
        finish = time.perf_counter()
        print(f'{vid_name} was downloaded')
```

```
        print(f'This takes {finish - start } sec to download')

t1 = time.perf_counter()

with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
# Run up to 5 downloads at the same time using background threads
    executor.map(download_video, vid_url)

t2 = time.perf_counter()
print(f'Finished in {t2-t1} seconds')
```

```
flash-wave-surge-particles-199581.mp4 was downloadedintro-circles-
flash-wave-231786.mp4 was downloaded
This takes 0.02718852099860669 sec to download
intro-wave-flash-light-217642.mp4 was downloaded
This takes 0.02903616600087844 sec to download

This takes 0.029718752000917448 sec to download
praya-de-paradiso-drone-cinematic-35540.mp4 was downloaded
This takes 0.022292486999504035 sec to download
woman-sample-laboratory-quality-197485.mp4 was downloaded
This takes 0.02924099899973953 sec to download
exploration-galaxy-generation-ship-130414.mp4 was downloaded
This takes 0.028380747000483098 sec to download
spiral-particles-wave-intro-217490.mp4 was downloaded
This takes 0.10694651200174121 sec to download
clock-time-minutes-green-screen-149068.mp4 was downloaded
This takes 0.03199990600114688 sec to download
sphere-space-spacecraft-130108.mp4 was downloaded
This takes 0.031081427001481643 sec to download
explosion-fireworks-flash-start-175299.mp4 was downloaded
This takes 0.16563714400035678 sec to download
information-window-information-plate-151346.mp4 was downloaded
This takes 0.02784815199993318 sec to download
sparks-particles-sample-background-212966.mp4 was downloaded
This takes 0.16409240000211867 sec to download
red-kaleidoscope-sample-background-108069.mp4 was downloaded
This takes 0.025551343998813536 sec to download
laboratory-glassware-microbiology-262189.mp4 was downloaded
This takes 0.09963720299856504 sec to download
purple-green-lines-abstraction-124825.mp4 was downloaded
This takes 0.03465541200057487 sec to download
circles-kaleidoscope-sample-124140.mp4 was downloaded
This takes 0.029297196000698023 sec to download
microscope-medical-laboratory-5445.mp4 was downloaded
This takes 0.18080360499880044 sec to download
particles-abstract-beautiful-bokeh-34965.mp4 was downloaded
This takes 0.09879496700159507 sec to download
Finished in 0.29975683500015293 seconds
```

This code optimizes the video downloader by using a ThreadPoolExecutor with a maximum of 5 concurrent workers to download the URLs in parallel. Since downloading files is an I/O-bound task, fetching 5 videos simultaneously drastically cuts down the overall execution time compared to downloading them one by one

TASK 3. Multithreading Priority Queue - Representative Diagram (conventional or non-conventional)

CODE:

```
import queue
import threading
import time
thread_exit_Flag = 0
class sample_Thread (threading.Thread):
   def __init__(self, threadID, name, q):
      threading.Thread.__init__(self)
      self.threadID = threadID
      self.name = name
      self.q = q
   def run(self):
      print ("initializing " + self.name)
      process_data(self.name, self.q)
      print ("Exiting " + self.name)
# helper function to process data
def process_data(threadName, q):
   while not thread_exit_Flag:
      queueLock.acquire()
      if not workQueue.empty():
         data = q.get()
         queueLock.release()
         print ("% s processing % s" % (threadName, data))
      else:
         queueLock.release()
         time.sleep(1)
thread_list = ["Thread-1", "Thread-2", "Thread-3"]
name_list = ["A", "B", "C", "D", "E"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
threadID = 1
# Create new threads
for thread_name in thread_list:
   thread = sample_Thread(threadID, thread_name, workQueue)
   thread.start()
   threads.append(thread)
   threadID += 1
# Fill the queue
queueLock.acquire()
for items in name_list:
   workQueue.put(items)
queueLock.release()
# Wait for the queue to empty
while not workQueue.empty():
```

```
   pass
# Notify threads it's time to exit
thread_exit_Flag = 1
# Wait for all threads to complete
for t in threads:
   t.join()
print ("Exit Main Thread")
```

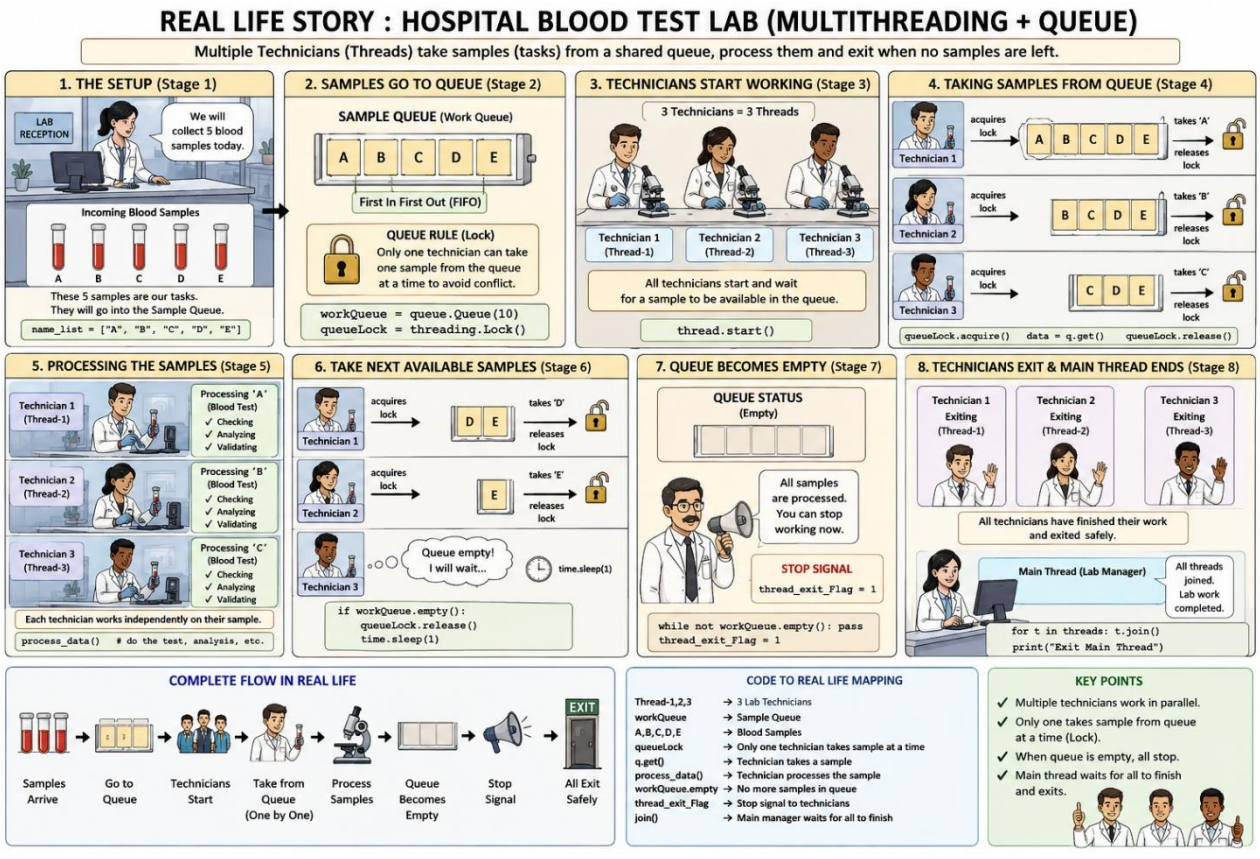
This code demonstrates multithreading with a shared queue, where multiple worker threads process tasks simultaneously. It is useful for simulating real-world systems such as order processing, task scheduling, and multiple workers handling jobs from a common queue.

[1]:
```python
import queue
import threading
import time
thread_exit_Flag = 0
class sample_Thread (threading.Thread):
   def __init__(self, threadID, name, q):
      threading.Thread.__init__(self)
      self.threadID = threadID
      self.name = name
      self.q = q
   def run(self):
      print ("initializing " + self.name)
      process_data(self.name, self.q)
      print ("Exiting " + self.name)
# helper function to process data
def process_data(threadName, q):
   while not thread_exit_Flag:
      queueLock.acquire()
      if not workQueue.empty():
         data = q.get()
         queueLock.release()
         print ("% s processing % s" % (threadName, data))
      else:
         queueLock.release()
         time.sleep(1)
thread_list = ["Thread-1", "Thread-2", "Thread-3"]
name_list = ["A", "B", "C", "D", "E"]
queueLock = threading.Lock()
workQueue = queue.Queue(10)
threads = []
threadID = 1
# Create new threads
for thread_name in thread_list:
   thread = sample_Thread(threadID, thread_name, workQueue)
   thread.start()
   threads.append(thread)
   threadID += 1
# Fill the queue
```

```
   threadID += 1
# Fill the queue
queueLock.acquire()
for items in name_list:
   workQueue.put(items)
queueLock.release()
# Wait for the queue to empty
while not workQueue.empty():
   pass
# Notify threads it's time to exit
thread_exit_Flag = 1
# Wait for all threads to complete
for t in threads:
   t.join()
print ("Exit Main Thread")
```

```
initializing Thread-1
initializing Thread-2
initializing Thread-3
Thread-2 processing AThread-3 processing B
Thread-3 processing C
Thread-3 processing D
Thread-3 processing E
Exiting Thread-1

Exiting Thread-2
Exiting Thread-3
Exit Main Thread
```



TASK 4. Practice printing MainThread name and worker threads

CODE:

```
from threading import *

def display():
    for i in range(10):
        print('Child Thread')

# Creation of Thread object to Execute Display
t = Thread(target=display)
t.start()

for i in range(10):
    print('Main Thread')
```

It shows the main thread and child thread running concurrently and printing their messages.

```
from threading import *

def display():
    for i in range(10):
        print('Child Thread')

# Creation of Thread object to Execute Display
t = Thread(target=display)
t.start()

for i in range(10):
    print('Main Thread')
```

```
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
```

CODE:

```
from threading import *

def display():
    print('Child Thread Name:', current_thread().getName())
    print('')

t = Thread(target=display)

t.start()

print('Main Thread Name:', current_thread().getName())
```

It helps understand how to identify and differentiate threads using current_thread().getName().

```
from threading import *

def display():
    print('Child Thread Name:', current_thread().getName())
    print('')

t = Thread(target=display)

t.start()

print('Main Thread Name:', current_thread().getName())
```

```
Child Thread Name: Thread-7 (display)

Main Thread Name: MainThread
C:\Users\rajav\AppData\Local\Temp\ipykernel_41132\1243201741.py:4: DeprecationWarning: getName() is deprecated, get the name attribute instead
  print('Child Thread Name:', current_thread().getName())
C:\Users\rajav\AppData\Local\Temp\ipykernel_41132\1243201741.py:11: DeprecationWarning: getName() is deprecated, get the name attribute instead
  print('Main Thread Name:', current_thread().getName())
```

CODE:

```
from threading import *

class MyThread(Thread):
    def run(self):
        for i in range(10):
            print('Child Thread')

t = MyThread()
t.start()       # triggers the run() method in a separate thread.

for i in range(10):     # Both loops execute concurrently
    print('Main Thread')

# The output of Child Thread and Main Thread will be
# interleaved, depending on the OS thread scheduler.
```

This code demonstrates creating a custom thread by inheriting from the Thread class and overriding the run() method. It shows the Main Thread and Child Thread executing concurrently, so their outputs may appear interleaved.

```
from threading import *

class MyThread(Thread):
    def run(self):
        for i in range(10):
            print('Child Thread')

t = MyThread()
t.start()       # triggers the run() method in a separate thread.

for i in range(10):     # Both loops execute concurrently
    print('Main Thread')

# The output of Child Thread and Main Thread will be
# interleaved, depending on the OS thread scheduler.
```

```
Child ThreadMain Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread

Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
```

CODE:

```
import threading
import time

def worker():
    for i in range(5):
        print("Working thread:", threading.current_thread().name)
        time.sleep(1)

print("Main thread:", threading.current_thread().name)

t = threading.Thread(target=worker, name="WorkerThread")
t.start()

for i in range(5):
    print("Main thread working:", threading.current_thread().name)
```

time.sleep(1)

t.join()

This code demonstrates multithreading by running a worker thread and the main thread simultaneously. It also shows thread names, uses sleep() to simulate work, and join() to make the main program wait for the worker thread to finish.

```python
import threading
import time

def worker():
    for i in range(5):
        print("Working thread:", threading.current_thread().name)
        time.sleep(1)

print("Main thread:", threading.current_thread().name)

t = threading.Thread(target=worker, name="WorkerThread")
t.start()

for i in range(5):
    print("Main thread working:", threading.current_thread().name)
    time.sleep(1)

t.join()
```

```
Main thread: MainThread
Working thread: WorkerThread
Main thread working: MainThread
Working thread:Main thread working: MainThread
 WorkerThread
Main thread working: MainThread
Working thread: WorkerThread
Working thread:Main thread working: MainThread
 WorkerThread
Main thread working: MainThread
Working thread: WorkerThread
```